

Temporal knowledge graph

Revised formal definition: OWL 2 DL representation,
primitive points and intervals, and temporal query semantics

v2 working specification, 15 September 2026

Status. This revision develops the four-page *Temporal Knowledge Graph: Formal definition*. It fixes the representation to OWL 2 DL, introduces disjoint primitive time points and intervals, and specifies temporal situations, occurrence, evidence selection and fixed-witness matching. Orange **OPEN** boxes identify decisions required before the corresponding capability can be operationalized. Section 13 collects the decisions and their acceptance evidence. The accompanying OWL files provide a checked core and a small example; the complete temporal query engine remains to be implemented.

1 Representation and execution contract

The primary knowledge base is a finite OWL 2 DL ontology

$$K_{\text{OWL}} = O \cup A,$$

where O is the schema and its pinned import closure, and A contains the accepted instance axioms for one declared evidence snapshot. Its models are ordinary OWL interpretations $\mathcal{J}$ with an object domain $\Delta^{\mathcal{J}}$, a data domain and interpretations of classes, properties and individuals. Entailment is OWL 2 Direct Semantics:

$$K_{\text{OWL}} \models_{\text{OWL}} \alpha \iff \forall \mathcal{J} \in \text{Mod}_{\text{OWL}}(K_{\text{OWL}}), \mathcal{J} \models \alpha.$$

Time is represented by individuals and explicit relations. A single interpretation describes an entire history. Changing occupancy, measurements and role use are represented through situations and process tokens. This replaces the original family of interpretations $\mathcal{I}_t$ with an OWL-native representation.

For temporal queries the application additionally supplies

$$\mathcal{P} = (K_{\text{OWL}}, E, \rho, \Gamma, \mathcal{Q}, \mathcal{R}),$$

where E is a finite evidence manifest and eligible witness set; ρ maps named temporal objects to a finite coordinate interface; Γ is a finite temporal constraint system; $\mathcal{Q}$ specifies supported queries; and $\mathcal{R}$ specifies optional relaxation. These components have separate contracts:

Component	Meaning	Responsible operation
OWL ontology	Classes, disjointness, identity, property restrictions, instance entailment	OWL 2 DL reasoning
Temporal interface	Endpoint order, coincidence, elapsed durations, uncertain coordinates	Explicit arithmetic/constraint evaluation
Evidence manifest	Eligible source-backed witnesses and an evidence-as-of snapshot	Versioned selection and validation
Query contract	Certain/possible fixed bindings, costs and answer projection	Finite binding search and solver calls

Annotations preserve provenance labels; logical consequences use object/data properties or ordinary axioms. OWL annotations carry no truth conditions under Direct Semantics [1]. The term “temporal entailment” below is qualified as either an OWL consequence or a consequence of Γ .

OPEN Q1: Execution boundary. May the application use an external temporal constraint evaluator while keeping its ontology in OWL 2 DL? This specification proposes that architecture. If every temporal answer must instead be obtained solely by an OWL reasoner, the supported service must be restricted to supplied qualitative facts, OWL consequences and explicitly compiled finite results. General endpoint subtraction, comparisons between two values and full temporal constraint solving require an additional operation. Decide this before selecting the engine.

2 Primitive time points and intervals

Let $P = \text{TimePoint}^{\mathcal{I}}$ and $I = \text{TimeInterval}^{\mathcal{I}}$ be two independently interpreted classes. An interval is an individual with endpoint relations. A coordinate assignment later provides a representation of its extent. The core axioms are

$$\begin{aligned} \text{TemporalEntity} &\equiv \text{TimePoint} \sqcup \text{TimeInterval}, \\ \text{TimePoint} \sqcap \text{TimeInterval} &\sqsubseteq \perp, \\ \text{TimeInterval} &\sqsubseteq (= 1 \text{ hasBeginning}.\text{TimePoint}) \sqcap (= 1 \text{ hasEnd}.\text{TimePoint}). \end{aligned}$$

Both endpoint properties have domain `TimeInterval` and range `TimePoint`, are functional, and are disjoint as properties. Consequently, an interval's beginning and end are distinct individuals. In the OWL file:

```
FunctionalObjectProperty(:hasBeginning)
FunctionalObjectProperty(:hasEnd)
DisjointObjectProperties(:hasBeginning :hasEnd)
```

Time points have zero extent in the coordinate interpretation. Intervals have positive extent in the metric profile of Section 3. Distinctness of endpoints follows in OWL; their strict temporal order is checked by the temporal evaluator. The core permits empty extensions of these classes; actual examples supply instances. It asserts neither an interval between every pair of points nor density of the object domain.

Identity and coincidence. Different IRIs may denote the same OWL individual. Conversely, two distinct point individuals may receive the same coordinate. Two distinct interval individuals may have coincident extents. The baseline keeps temporal coincidence separate from `owl:sameAs`; it supplies no endpoint key. This accommodates different temporal descriptions while preserving process identity and assertion provenance separately.

OPEN Q2: Identity of temporal individuals. Are points identical whenever they are coincident in a reference frame? Are intervals identical whenever both endpoints coincide? The baseline treats coincidence as a temporal relation and requires explicit identity decisions. Before ingestion, define the canonicalization policy for patients, processes, roles and temporal individuals, and how equality inferred by OWL updates the temporal interface. Source-record identity must remain independently traceable.

OWL-Time alignment. Proposed class mappings are

$$\text{TimePoint} \sqsubseteq \text{time:Instant}, \quad \text{TimeInterval} \sqsubseteq \text{time:ProperInterval}.$$

Endpoint properties can be mapped to `time:hasBeginning` and `time:hasEnd`. OWL-Time's broader `time:Interval` is deliberately not equated with our positive-duration interval class. The standalone checked core uses local terms and has no ontology imports. These mappings remain an integration module to be reviewed against a pinned OWL-Time version [3].

3 Coordinates, extent and temporal constraints

For a query run, let P_N and I_N be finite sets of named, identity-normalized points and intervals admitted by the evidence manifest. Every admitted interval has explicitly identified, entailed unique endpoints $b(i), e(i) \in P_N$. An existential endpoint required by OWL is sufficient for ontology consistency; an eligible metric-query witness needs named endpoint bindings.

The metric profile uses one normalized reference axis with rational coordinates in a fixed elapsed-time unit. Let U be a finite set of auxiliary clock/anchor variables and $X = P_N \cup U$, with U disjoint from the point names. Define

$$\theta : X \longrightarrow \mathbb{Q}, \quad \Omega = \{\theta : \theta \models \Gamma\}.$$

Each admitted interval contributes $\theta(b(i)) < \theta(e(i))$ to Γ . This is a coordinate model of the primitive interval, not its OWL identity. Define its realization and that of a point by

$$\text{ext}_\theta(i) = [\theta(b(i)), \theta(e(i))], \quad \text{ext}_\theta(p) = \{\theta(p)\}.$$

The half-open convention governs membership in an asserted extent. The end is still the interval's boundary point. Using the same point individual for one interval's end and another's beginning is permitted.

For $s_i = \theta(b(i))$ and $e_i = \theta(e(i))$, the evaluator uses:

Condition	Exact coordinate meaning
Point before (p, q)	$\theta(p) < \theta(q)$
Interval before (i, j)	$e_i < s_j$
Meets (i, j)	$e_i = s_j$
During (i, j)	$s_j < s_i \wedge e_i < e_j$
Overlaps (i, j)	$s_i < s_j < e_i < e_j$
Starts (i, j)	$s_i = s_j \wedge e_i < e_j$
Finishes (i, j)	$s_j < s_i \wedge e_i = e_j$
Coincides (i, j)	$s_i = s_j \wedge e_i = e_j$
Duration (i)	$e_i - s_i$
Gap (i, j)	$s_j - e_i$

The six directed interval relations and their inverses, plus coincidence, partition pairs of realized proper intervals. This partition is enforced by the coordinate interpretation. The OWL core declares relation types and inverses; general endpoint arithmetic and exhaustiveness are properties of the evaluator.

The initial constraint grammar is a conjunction of rational difference constraints

$$x - y \leq c \quad \text{or} \quad x - y < c,$$

where x, y are named coordinate variables or a fixed zero anchor, and $c \in \mathbb{Q}$. Equalities expand to two inequalities. Known timestamps supply equalities; uncertain timestamps supply bounds. Shared clock errors use shared variables, preserving correlations. Arbitrary disjunctions of interval relations require a separately declared solver extension.

Clock mapping. ρ records the origin, unit and mapping used for each admitted point. A known rational affine conversion can be performed before constraint solving. An unknown shared additive offset can be represented by a variable. Unknown clock rates, calendar arithmetic and nonlinear transformations require a richer constraint fragment. A bound on an event time denotes uncertainty about that point; an interval-valued state denotes validity throughout its extent.

OPEN Q3: Clocks and precision. Choose the authoritative clock, timezone normalization, elapsed-time unit, treatment of daylight-saving/leap-second records, measurement precision and uncertain clock offsets. The baseline uses exact rational arithmetic after conversion. Declare which mappings preserve the difference-constraint fragment and how unmappable timestamps are reported. An unavailable mapping makes the comparison unsupported for that witness.

4 Process occurrence and incomplete extents

A process token identifies a particular occurrence, independently of the assertion that documents it. An accepted complete temporal location is represented by

$$\text{exactExtent}(p, z), \quad p : \text{Process}, \quad z : \text{TemporalEntity}.$$

The property is functional. The following OWL classes distinguish point and interval occurrences:

$$\begin{aligned} \text{InstantaneousProcess} &\equiv \text{Process} \sqcap \exists \text{exactExtent}.\text{TimePoint}, \\ \text{ExtendedProcess} &\equiv \text{Process} \sqcap \exists \text{exactExtent}.\text{TimeInterval}. \end{aligned}$$

Their disjointness follows from the disjoint temporal classes and functionality. The general **Process** class has no minimum cardinality on **exactExtent**; a process can be documented before its full extent is known.

For an admitted process, the temporal query contract interprets `exactExtent` as its complete occurrence extent. An assertion that an instantaneous process occurred somewhere between two bounds uses one point with constrained coordinates. An uncertain duration uses an interval with constrained endpoints. Merely knowing that a process lies inside a time window supplies a containment constraint on its unknown extent. Label that record as a window constraint; using the window as `exactExtent` would assert a different claim.

Repeated administrations use distinct process tokens. A source timestamp with unspecified meaning remains a timestamp claim until the source mapping identifies it as a beginning, end, point occurrence, observation or recording time.

OPEN Q4: Ongoing and disconnected processes. Which process types require point extents, bounded intervals, unknown ends or multiple separated components? The implemented core proposed here covers points and bounded proper intervals. It leaves an ongoing process’s complete extent unasserted until an approved representation is chosen. If recurring/disconnected extent or unbounded continuation is required, define its class, membership semantics and query operations before admitting such witnesses. A guessed end time is not an acceptable completion policy.

5 Validity through temporal situations

Time-dependent relations are represented by typed situation individuals. For example, a bed occupancy situation u has an occupant p , a bed b , and a validity extent z :

$$\begin{aligned} \text{BedOccupancySituation} &\sqsubseteq \text{TemporalSituation}, \\ \text{TemporalSituation} &\sqsubseteq (= 1 \text{ validExtent.TemporalEntity}), \\ \text{BedOccupancySituation} &\sqsubseteq (= 1 \text{ hasOccupant.Patient}) \sqcap (= 1 \text{ occupiedBed.Bed}). \end{aligned}$$

The three properties have the corresponding domains/ranges and are functional. Their assertions describe a situation across the entire stored history. They supply no timeless `occupiesBed(patient, bed)` shortcut.

A query-side abbreviation makes the truth-throughout convention explicit. For an OWL model $\mathcal{J}$ and a coordinate assignment θ , define supported occupancy at coordinate t by

$$\text{Occupies}_N(p, b, t; \mathcal{J}, \theta) \iff \exists u, z \in N \quad \mathcal{J} \models \text{BedOccupancySituation}(u) \wedge \text{hasOccupant}(u, p) \wedge \text{occupiedBed}(u, b) \wedge \text{validExtent}(u, z), \quad t \in \text{ext}_\theta(z).$$

Here N is the admitted named interface, and variables range over names of the appropriate sort. This definition is a query macro over reified situations. OWL entailments continue to concern situation classes and properties. Interval validity is exact as a positive extent of support; it is silent about occupancy outside that extent. Repeated situations can support separated periods. A single observation supports a point unless a source-backed abstraction supplies an interval.

Ontology lifting. An atemporal domain axiom about patients or relations needs an explicit representation policy before it is applied to temporal situations. For example, whether an occupancy restriction means “one bed at each time” or “one bed in the whole history” changes its translation. Per-situation functionality states one bed for each situation. Detecting two conflicting situations that overlap in time requires the temporal query/validation layer. General time-indexed ontology entailment is outside the declared OWL core.

OPEN Q5: State vocabulary and conflict semantics. Choose the domain predicates to represent as situations and provide a reviewed template for each, including class membership, numeric values and negative assertions where needed. Specify coverage/interpolation assumptions for sustained states, the treatment of conflicting overlapping situations, and whether source absence is ever treated as false. Approve temporal translations of domain axioms separately from ordinary OWL imports.

6 PRO participation with explicit historical scope

The process–role–object pattern describes participation in a particular capacity. A process a links to a role token r , and that token links to its bearer p . The local core uses `hasParticipant` and `isFeatureOf`, with `PatientRole` and `AdministeredDrugRole` subclasses of `Role`. A token denotes the particular historical role

use intended by the record. Changing capacities are represented through distinct role uses or a reviewed role-situation template.

The application class definition is

$$\text{AntibioticAdministration} \equiv \text{DrugAdministration} \sqcap \exists \text{hasParticipant}. (\text{AdministeredDrugRole} \sqcap \exists \text{isFeatureOf}. \text{Antibiotic}).$$

The property chain

$$\text{hasParticipant} \circ \text{isFeatureOf} \sqsubseteq \text{hasParticipant}$$

entails bearer participation. Queries retain the role token and its class because bearer participation alone carries less information about capacity. Both class and chain axioms are ordinary OWL axioms.

The local names reproduce this pattern from SULO, whose inspected ontology identifies itself as v0.2.14 [4]. The checked core is a standalone local vocabulary, with a narrower local domain of `isFeatureOf` restricted to roles. It should be aligned by reviewed subproperty/subclass mappings; equating the local property with SULO’s general feature relation would impose the local restriction on other SULO features.

Temporal scope of participation. These assertions describe historical participation in the named process. They do not select a current-time patient status. If the task needs the exact portion of a process during which a bearer participates, add a participation situation with an explicit validity extent and specify its containment in the process extent. This is an extension decision under Q5.

OPEN Q6: Ontology imports and biomedical mappings. Pin the SULO, OWL-Time and domain ontology versions; approve local-to-source mappings, role-token granularity, drug concept typing and administration/collection distinctions. Run OWL 2 DL profile and consistency checks on the complete import closure. The accompanying standalone-core check establishes a baseline only; the selected imports and mapping axioms require their own validation.

7 Evidence, assertion identity and revision

A source record, an assertion about the world, and the process or situation described have separate identities. The core includes `SourceRecord` and `AssertionRecord`, with `supportedBy`, `aboutEntity` and `supersedes`. These links represent provenance structure. Logical acceptance is supplied by a versioned manifest whose entries associate each assertion identifier with an explicit finite bundle of ground OWL axioms and temporal constraints. An IRI denoting a class or property in a record is interpreted by that manifest; OWL punning supplies no generic evaluation of reified triples.

For a source availability cutoff κ , revision policy π , and mapping/ontology version bundle ν , define

$$S_{\kappa, \pi, \nu} = \text{Select}(E, \kappa, \pi, \nu), \quad K_{\kappa, \pi, \nu} = O_\nu \cup \bigcup_{a \in S_{\kappa, \pi, \nu}} \text{axioms}_\nu(a).$$

Temporal constraints are selected from the same entries. The manifest additionally records source collection/observation time, first availability, system recording time, revision links, and which named process/role/situation witnesses are eligible. These dates have distinct fields and meanings. The query cutoff is evaluated against source availability, with an explicit policy for unknown availability.

Selection creates a new immutable reasoning snapshot. Retracting an assertion in a later snapshot removes its contribution; prior snapshots remain addressable. A `supersedes` edge in OWL alone has no retraction operation. Duplicate support for the same axiom must survive removal of only one source.

OPEN Q7: Evidence selection and historical reproducibility. Define `Select`, including admissible source types, missing availability times, contradictory sources, corrections, duplicate support and eligible witness extraction. Decide whether an as-of analysis uses the ontology/mapping available then or a declared retrospective version. Store both axes if querying historical validity and recording history is required. Operational bitemporal behavior depends on this selection contract and its tests.

8 Finite query language and answer semantics

The initial query language comprises a finite conjunction

$$\Psi(\mathbf{x}) = \bigwedge_{k=1}^m \alpha_k(\mathbf{x}) \wedge C(\mathbf{x}),$$

where α_k is a named class assertion or object/data property assertion after binding, and C is a conjunction of supported temporal difference constraints. All process, role, situation and endpoint witnesses are explicitly bound to eligible named objects. Equality is handled through the identity-normalized interface. More expressive class expressions can be queried if their ground class assertions are supported by the OWL reasoner; hidden query existential variables, aggregation, recursion and unrestricted negation require an extension of this contract.

Let H_E be the finite set of eligible bindings; $\mu \in H_E$ fixes *all* witnesses. Eligibility is a source-manifest property, while class and property membership are checked by OWL entailment. They are separate conditions.

8.1 Separated baseline

The operational baseline admits a binding only when

$$\text{Supported}(\mu) \iff \mu \in H_E \wedge \bigwedge_k K_{\text{OWL}} \models_{\text{OWL}} \alpha_k[\mu].$$

The coordinate interface is fixed after OWL identity normalization. Each binding includes the named temporal objects and their entailed unique endpoints; the validator rejects ambiguity in this interface. The compiler reads only named, uniquely established endpoint links and the declared constraint records. OWL axioms may infer domain types and participation. In the baseline, they introduce no additional arithmetic constraints or time-dependent changes to these structural bindings. Any temporal qualitative assertions used as constraints are compiled once into Γ through the declared relation table. The initial compiler accepts positive named temporal relation assertions and their entailed inverse forms. Arbitrary negative, disjunctive or class-conditioned temporal constraints require Q8’s extension review. Coordinate-derived results are returned as evaluator conclusions; the baseline has no feedback step that adds them to K_{OWL} and triggers new domain inferences.

After checking OWL consistency and $\Omega \neq \emptyset$, define

$$\begin{aligned} \text{Certain}(\mu) &\iff \text{Supported}(\mu) \wedge \forall \theta \in \Omega \ C[\mu, \theta], \\ \text{Possible}(\mu) &\iff \text{Supported}(\mu) \wedge \exists \theta \in \Omega \ C[\mu, \theta]. \end{aligned}$$

“Possible” here means temporal compatibility of an ontologically supported, source-backed binding. This tightens the original document’s unrestricted possible-model criterion, which also allowed models to supply missing positive ontology facts. A separate logical-possibility service could retain that broader criterion, but must label the evidential difference.

A patient is returned as certain when at least one *fixed* eligible binding for that patient is certain. Formally the witness policy is $\exists \mu \forall \theta$, evaluated before projecting the patient identifier. For two candidate times that can be (12, 36) or (36, 12), a 24-hour query has some qualifying candidate in every assignment, yet neither fixed candidate is certain. The policy therefore preserves explainable witness identity.

8.2 Solver contract and integration limit

For each supported binding,

$$\text{Possible}(\mu) \iff \text{SAT}(\Gamma \wedge C[\mu]), \quad \text{Certain}(\mu) \iff \text{UNSAT}(\Gamma \wedge \neg C[\mu]).$$

If $C = \bigwedge_j c_j$, certainty can be checked by testing $\Gamma \wedge \neg c_j$ separately for each conjunct. This keeps each call in rational difference constraints. Strict inequalities are represented exactly. All shared constraints remain in each call; independent marginal bounds can lose correlations.

The baseline combines OWL support and temporal checks by a specified application query operation. Richer bridges that condition class membership on duration or introduce temporal constraints through inferred

facts require a new compilation or joint-model contract. In that extension, a compatible world would be $(\mathcal{J}, \theta)$ satisfying OWL, Γ and every bridge condition; sound independent evaluation would need a proof for those bridges.

OPEN Q8: Supported engine and bridge fragment. Choose the OWL reasoner, permitted imports/axioms, query grammar, exact temporal solver, and finite interface validation rules. Decide whether all necessary queries fit the separated baseline. If additional bridges are required, specify them and prove that compilation preserves consistency and query answers. Termination of OWL reasoning alone leaves this integration obligation open.

Result statuses. Distinguish certain, possible-only, impossible for a supported binding, unsupported evidence, OWL inconsistency, temporal inconsistency, and execution failure/timeout. A complete empty answer set requires completed candidate enumeration. Missing data, unsupported syntax and timeout are execution/evidence statuses, not temporal uncertainty results.

9 Worked administration–collection pattern

Bind patient p , administration a , collection b , patient role tokens r_a, r_b , their extents i_a, i_b , and all four endpoints. Require the following ontology atoms:

$$\begin{aligned} & \text{AntibioticAdministration}(a) \wedge \text{BloodSpecimenCollection}(b) \\ & \wedge \text{PatientRole}(r_a) \wedge \text{hasParticipant}(a, r_a) \wedge \text{isFeatureOf}(r_a, p) \\ & \wedge \text{PatientRole}(r_b) \wedge \text{hasParticipant}(b, r_b) \wedge \text{isFeatureOf}(r_b, p) \\ & \wedge \text{exactExtent}(a, i_a) \wedge \text{exactExtent}(b, i_b), \end{aligned}$$

plus their entailed endpoint links and interval types. This example retains the original completion-to-start criterion:

$$0 < \theta(b(i_b)) - \theta(e(i_a)) \leq 48 \text{ hours.}$$

An administration over $[0, 1)$ and collection over $[25, 25.1)$ gives a 24-hour gap and a certain match when those coordinates are exact. If the collection start lies in $[48, 50]$ and the administration ends at 1, the gap lies in $[47, 49]$; the binding is possible-only. Both examples are illustrative temporal calculations.

OPEN Q9: Clinical endpoint and boundary convention. Confirm whether each query measures from administration start, completion or another landmark, and whether equality at zero is allowed. The retained condition excludes a collection at the exact completion point. Approve the clinical pattern and boundary examples before treating the matcher as a validated cohort definition. Source timestamp precision must enter Q3’s uncertainty policy.

10 Controlled relaxation

A relaxation r belongs to a finite approved catalogue and modifies named query conditions while preserving declared hard conditions. It contains its concept substitutions, temporal widening and budget consumption. The original query remains available for exact matching. For a binding $\mu \in H_E$ and one fixed relaxation r , require OWL entailment of the ontology atoms after the approved substitutions, written $\text{Supported}_r(\mu)$. Source eligibility and every protected ontology condition remain mandatory. Let $c_r(\mu, \theta) \geq 0$ be the declared cost, with $+\infty$ assigned to a binding that fails this revised support test. Robust acceptance requires that the relaxed hard conditions hold for every $\theta \in \Omega$. Set cost to $+\infty$ for any violating assignment. A patient-level score is

$$\text{Score}(p) = \min_{\substack{\mu \in H_E(p) \\ r \in \mathcal{R}}} \sup_{\theta \in \Omega} c_r(\mu, \theta),$$

subject to the relaxation budget. Both the binding and relaxation are chosen before the uncertain times. Using sup covers open feasible sets whose worst-case cost is approached without attainment. The exact query is the zero-relaxation member. A separate policy must specify when a finite score qualifies for return.

OPEN Q10: Relaxation catalogue and optimization. Approve allowed concept replacements, metric widening, hard constraints, cost functions, budgets and tie-breaking. Choose a cost fragment with a complete optimizer, or restrict the catalogue to constant-cost query modifications for which the robust feasibility checks suffice. Keep possible-only ranking separate from robust acceptance.

11 OWL 2 DL compliance and operational gates

The accompanying files use only standard OWL 2 DL constructs. Endpoint, extent and situation-slot properties remain simple, so their functionality and cardinality restrictions satisfy the relevant structural requirement. The PRO chain makes `hasParticipant` non-simple; the core uses it in existential restrictions and avoids functionality or cardinality axioms on it.

The core’s `pointBefore` is simple and asymmetric. Transitive temporal closure is assigned to the evaluator. Combining transitivity with asymmetry or irreflexivity on that same property violates OWL 2 DL’s simple-property restrictions. Full interval-algebra reasoning and numerical validation therefore have explicit application contracts [2].

OWL restrictions express model conditions. A required unnamed endpoint can satisfy an existential restriction. Functional properties can infer identity between two fillers. An ingestion validator must therefore check named required fields, source eligibility and explicit identity policy in addition to consistency.

Checks included with this revision. ROBOT v1.9.10 / OWLAPI profile checking and its bundled Hermit reasoner passed 18 checks for the standalone core/example. These include satisfiability of every named class in the example, PRO inference, disjointness, endpoint equality consequences, and the deliberately external endpoint-order condition. Exact outputs and the Java checker are in the accompanying validation directory. The source files are `temporal-core.ofn` and `example.ofn`; local namespace IRIs under `example.org` identify a draft. The application fragment described above has no end-to-end implementation in this deliverable.

Before deployment, the acceptance suite must cover:

1. Point/interval disjointness, endpoint distinction, point versus interval processes, PRO class inference, and identity consequences of functional properties.
2. Positive and negative temporal boundary cases; shared uncertain clocks; contradictory constraints; unknown mappings; and exact rational strict inequalities.
3. Fixed-witness versus projected certain membership; absent eligible source records; missing named endpoints; and anonymous ontology witnesses.
4. Corrections and duplicate support across snapshots; source availability cutoffs; mapping-version changes; and contradictions between accepted claims.
5. Query completeness and pruning against exhaustive enumeration on finite fixtures; relaxation costs against independently checked optima.

OPEN Q11: Operational acceptance and scale. Specify target data volume, supported query sizes, latency/resource budgets, completeness guarantees and timeout behavior. Select a representative source-backed benchmark and an independently checked reference evaluator. Passing OWL profile and toy consistency tests establishes ontology compatibility; the full service requires the query and evidence tests above.

12 Position in the six-axis classification

Axis	Revised commitment
O	Distinct primitive points and intervals; processes and temporal situations as individuals; explicit identity/co-incidence policy.
T	Qualitative relations in OWL; a declared rational metric coordinate profile for admitted named objects. Object-domain density and universal temporal order remain stronger optional commitments.
D	Situation validity and process occurrence; explicitly distinguished observation, availability and recording metadata. Bitemporal querying depends on the selection contract.
B	Situation and process individuals carry extents; assertion/source identifiers preserve evidence and revision separately.
A	Point and bounded interval extents; correlated coordinate uncertainty. Ongoing/disconnected extents await Q4.
S	OWL 2 DL entailment plus a declared finite evidence-grounded temporal query operation, with optional robust relaxation.

The primary OWL profile is a KG about time and temporally situated domain facts. Interval-validity query behavior comes from the explicit situation contract. Modal temporal logic and causal interventions would add further semantics. Compared with the original document, the ontology itself now uses standard OWL interpretations; temporal query claims identify their additional evaluator explicitly.

13 Decision register: open before operationalization

The baseline choices above make the proposal concrete. The following decisions remain open; their scope column identifies the capability they block. An optional capability may be deferred explicitly while a smaller supported service proceeds.

ID	Decision / blocked capability	Proposed baseline	Evidence required to close
Q1	Execution boundary / all metric queries	OWL 2 DL ontology with separate temporal evaluator	Agreed service contract naming OWL and evaluator conclusions.
Q2	Identity / ingestion and temporal joins	Explicit canonical identities; co-incidence separate	Equality/canonicalization rules and merge/conflict examples.
Q3	Clocks / numeric comparisons	One normalized axis; exact rational constraints	Source clock mappings, uncertainty rules and conversion fixtures.
Q4	Process extents / ongoing or disconnected histories	Points and bounded proper intervals	Type-specific extent policy; tests for admitted extensions or explicit exclusion.
Q5	State templates / time-varying domain queries	Typed positive situations with explicit validity	Reviewed predicate templates, overlap/conflict and observation-to-state rules.
Q6	Imports and mappings / biomedical integration	Standalone checked core; reviewed alignment module	Pinned imports, mapping rationale, full-closure DL and consistency checks.
Q7	Evidence selection / trustworthy as-of answers	Immutable selected snapshots with eligible named witnesses	Executable Select specification; correction, duplicate-support and cutoff tests.
Q8	Logic/solver interface / complete query evaluation	Finite ground ontology checks plus difference constraints	Compiler/interface proof, solver choice and answer-preservation fixtures.
Q9	Clinical query / application validity	Strict completion-to-start gap up to 48 hours	Approved landmark/boundary definition and clinical examples.
Q10	Relaxation / robust ranking	Fixed binding and relaxation; worst-case cost	Approved catalogue/budgets; supported optimizer and reference cost tests.
Q11	Scale and acceptance / production operation	Explicit result statuses and complete finite baseline	Benchmark, resource targets, reference answers and failure reporting.

References

- [1] Motik B, Patel-Schneider PF, Cuenca Grau B, eds. *OWL 2 Web Ontology Language: Direct Semantics*, second edition. W3C Recommendation, 2012. <https://www.w3.org/TR/owl2-direct-semantics/>.
- [2] Motik B, Patel-Schneider PF, Parsia B, eds. *OWL 2 Web Ontology Language: Structural Specification and Functional-Style Syntax*, second edition. W3C Recommendation, 2012. Especially section 11. <https://www.w3.org/TR/owl2-syntax/>.
- [3] Cox S, Little C, eds. *Time Ontology in OWL*. W3C Recommendation, 2017. <https://www.w3.org/TR/owl-time/>.
- [4] AIDAVA-DEV. *Simplified Upper Level Ontology*, inspected artifact reporting version 0.2.14, retrieved 15 September 2026. <https://github.com/AIDAVA-DEV/sulo>. The checked local core reproduces the PRO pattern with local terms; full import integration is an open decision.